Punto 05 — Diseño de software

Proyecto: Monitor de Consumo Eléctrico — EnergyMonitor **Ficha:** 3145556 — Análisis y Desarrollo de Software (ADSO), SENA CIES Regional Huila **Integrantes:** Juan Esteban Cortés Parra · Miguel Ángel García Artunduaga **Versión del documento:** 1.0

> **Nota de actualización (27/10/2026).** Este documento describe el diseño de la versión 1.0, cuando el backend era el monolito modular `energy-monitor-backend` (Spring Modulith). Ese diseño evolucionó: los siete contextos acotados son hoy **siete microservicios** detrás del API gateway `en-monitor-api-gateway` (8080) — seguridad 8082, dispositivos 8084, mediciones 8085, hogares 8086, notificaciones 8087, alertas 8088 y recomendaciones 8089 —, comunicados por REST interno (`/internal/**`) y eventos de RabbitMQ con outbox transaccional, sobre el mismo esquema MySQL compartido (37 tablas). El backend modular quedó retirado. Las capas hexagonales, los contextos y los patrones descritos aquí se conservan dentro de cada servicio; la arquitectura vigente, las rutas, los eventos y la configuración están en el manual técnico (secciones 9 a 13) y el modelo de datos vigente en `06-base-datos`.

1. Propósito del punto 05

El análisis de software (punto 03) resuelve **qué** debe hacer el sistema y **cómo debe verse**. El diseño de software responde a **cómo se construye**: qué estructura tiene el sistema por dentro, cómo se organizan sus datos, con qué patrones se resuelve cada problema recurrente y cómo se materializa el diseño visual en componentes reutilizables.

Este punto es el puente entre el análisis y la implementación. Cada decisión de diseño que aquí se documenta está respaldada por código existente en los tres repositorios del proyecto, no por descomposiciones meramente teóricas.

2. Documentos de este punto

Documento	Contenido	Pregunta que responde
`05.1-modelo-arquitectonico.md`	Estilo arquitectónico, capas, contextos acotados, componentes, despliegue, atributos de calidad y reglas verificables	**¿Cómo se organiza el sistema?**
`05.2-modelo-de-datos.md`	29 tablas, convenciones, referencias entre contextos, integridad, Liquibase y diferencias con el MER del punto 03	**¿Cómo se almacenan los datos?**
`05.3-modelo-de-clases.md`	Clases, interfaces, enumeraciones, relaciones y diferencias con el diagrama de clases del punto 03	**¿Cómo se comporta el sistema por dentro?**
`05.4-patrones-de-diseno.md`	18 patrones documentados con problema, aplicación, beneficio y costo	**¿Qué soluciones se aplicaron y por qué?**
`05.5-interfaces-graficas.md`	Sistema de diseño, paleta, tipografía, pantallas, componentes, interacción, accesibilidad e i18n	**¿Cómo se ve y cómo se usa?**

3. Resumen ejecutivo

Aspecto	Definición
Estilo arquitectónico	Cliente-servidor en capas + API gateway + 7 microservicios con arquitectura hexagonal (versión 1.0: monolito modular Spring Modulith, hoy retirado)
Contexto de presentación	React 19 + Vite 8 (web) y Expo SDK 57 (móvil), con sistema de diseño propio en ambos

Aspecto	Definición
Lógica	Java 21 + Spring Boot 4.1.1, organizada en **7 contextos acotados** con límites verificados por pruebas
Datos	MySQL 8.4, esquema compartido con **37 tablas** (29 de negocio en la versión 1.0, más outbox y registros pendientes), cada servicio dueño de las suyas, Liquibase y borrado lógico
Comunicación	API REST sin estado bajo `/api/v1` con JWT RS256; telemetría de dispositivos por MQTT
Patrones	18 patrones documentados, con énfasis en puertos y adaptadores, Observer, Strategy y Repository
Interfaz	Sistema de diseño propio con 32 tokens, 4 temas, 11 medidas tipográficas, 4 idiomas
Pruebas	116 archivos de prueba en el backend (6 niveles) · 13 suites en la web · 0 en el móvil

4. Estructura de los repositorios

Repositorio	Tecnología	Punto de entrada
`en-monitor-api-gateway` y siete `en-monitor-*-service`	Java 21 + Spring Boot 4.1.1 (gateway 4.0.8) + Maven	`*Application.java` (gateway 8080; servicios 8082 y 8084 a 8089)
`energy-monitor-backend` (retirado)	Java 21 + Spring Modulith	Ya no se ejecuta; conserva el firmware
`Energy-Monitor-application`	React 19 + Vite 8 (JavaScript)	`src/main.jsx` sobre `index.html` (puerto 5173)
`Energy-Monitor-Mobile`	Expo SDK 57 + React Native 0.86 (TypeScript)	`src/app/_layout.tsx`

5. Recursos gráficos

Archivo	Contenido
`assets/diagramas/MER.wsd`	Modelo entidad-relación en sintaxis PlantUML (artefacto del punto 03)
`assets/diagramas/Diagrama_clases.wsd`	Diagrama de clases en sintaxis PlantUML (artefacto del punto 03)

Los archivos se renderizan en https://www.plantuml.com/plantuml/uml o con el plugin de PlantUML para Visual Studio Code.

6. Relación con los demás puntos del proyecto

6.1 Punto 03 — Análisis de software (origen de este punto)

Artefacto del punto 03	Qué aporta al punto 05
Mockups (web y móvil)	Definen el aspecto visual de cada pantalla → `05.5-interfaces-graficas.md`
Mapas de navegación	Definen el árbol de pantallas → `05.5-interfaces-graficas.md`, secciones 4 y 5
Historias de usuario y casos de uso (punto 02)	Definen las funcionalidades → `05.1-modelo-arquitectonico.md`, sección 3
MER	Define el modelo de datos → `05.2-modelo-de-datos.md`
Diagrama de clases	Define el modelo de clases → `05.3-modelo-de-clases.md`

> **Los mockups viven en el punto 03.** Este punto los describe y los relaciona con el código, pero no los duplica.

6.2 Punto 02 — Especificación de requisitos

Requisito	Dónde se materializa
RF-SEC Gestión de Usuarios y Perfiles	Contexto `security` · pantallas de acceso y cuenta
RF-HOM Gestión de Hogares	Contexto `home` · panel y detalle del hogar

Requisito	Dónde se materializa
RF-HOM-08, RF-NOT-03, RF-CLI-05, RF-CLI-06 Umbrales, preferencias, idioma y colores	`home_thresholds` + `user_configuration` · pantalla de configuración
RF-MED, RF-ALE, RF-REC, RF-NOT Mediciones, alertas, recomendaciones y notificaciones	Contextos `measurement`, `alert`, `recommendation` · pestañas de consumo e historial
RF-DEV Gestión de Dispositivos (estado de conexión: RF-MED-04 a RF-MED-06)	Contexto `device` · pestaña de dispositivos y asistente de vinculación
RNF-SEG, RNF-REN, RNF-DIS, RNF-COM, RNF-MAN, RNF-I18, RNF-ESC, RNF-USA, RNF-INT	`05.1-modelo-arquitectonico.md`, sección 7

6.3 Punto 04 — Propuesta técnica

La propuesta técnica define **con qué** se construye el sistema. Este punto define **cómo** se estructura. Las decisiones técnicas tomadas en el punto 04 (lenguajes, frameworks, herramientas) se aplican aquí de forma concreta.

6.4 Puntos posteriores

Punto	Qué toma de este punto
06 — Base de datos	Diccionario de datos y scripts de base de datos derivados del modelo de datos
07 — Pruebas	Casos de prueba derivados de las reglas de integridad y de los límites de módulo
08 — Calidad y mejoras	Brechas registradas en las secciones 9 y 10 de cada documento
09 — Implementación	Trazabilidad entre los documentos de diseño y las tareas de construcción
10 — Manual técnico	Arquitectura, estructura de carpetas y configuración
12 — Manual de usuario	Pantallas, componentes y mensajes definidos en `05.5-interfaces-graficas.md`

7. Observaciones de control de versiones

7.1 Cambios entre el artefacto de diseño (punto 03) y el código implementado

Durante la construcción del software, el esquema y el modelo de clases evolucionaron. Este punto documenta **el estado vigente** y deja constancia de las diferencias en las secciones 9 (`05.2-modelo-de-datos.md`) y 10 (`05.3-modelo-de-clases.md`).

Documento	Sección	Contenido
`05.2-modelo-de-datos.md`	9	13 diferencias entre el MER del punto 03 y el esquema vigente
`05.3-modelo-de-clases.md`	10	8 diferencias entre el diagrama de clases del punto 03 y el código vigente

Diferencias más relevantes:

#	Diferencia	Impacto
1	El MER publicado no incluye todas las tablas; el esquema vigente tiene 29	Se añadieron `notification_preference` y `push_subscription`
2	La nota del diagrama de clases afirma que el patrón Strategy no está implementado	**Las tres clases sí existen** en `home/domain/model`
3	El diagrama de clases modela `RecommendationAnalyzer`	El código usa `RecommendationRules` (servicio de dominio)
4	`AlertType` tenía dos valores	El esquema vigente tiene cuatro

Acción recomendada: actualizar `MER.wsd` y `Diagrama_clases.wsd` en `05-diseno-software/ES/assets/diagramas/` antes de la entrega final.

7.2 Brechas de diseño registradas

Documento	Sección	Número de brechas
`05.2-modelo-de-datos.md`	10	5 decisiones pendientes
`05.4-patrones-de-diseno.md`	8	5 decisiones de no aplicar patrón
`05.5-interfaces-graficas.md`	7.2	6 brechas de accesibilidad
`05.5-interfaces-graficas.md`	10	10 trabajos pendientes de interfaz

Estas brechas están documentadas de forma deliberada: la guía del programa recomienda describir el software **tal como está construido**, sin ocultar lo que falta. Cada una lleva su recomendación de solución.

8. Cómo usar este punto

Si eres…	Empieza por…
Evaluador de la ficha	La sección 3 de este índice y la sección 7 de `05.1-modelo-arquitectonico.md`
Desarrollador nuevo en el proyecto	`5.1` → `5.4` → el código del módulo que te asigne
Diseñador de base de datos	`05.2-modelo-de-datos.md` y `05.3-modelo-de-clases.md`
Diseñador de interfaz	`05.5-interfaces-graficas.md` y los mockups del punto 03
Responsable del proyecto	Las secciones de brechas: 5.2 §10, 5.4 §8, 5.5 §7.2 y §10